



HACETTEPE UNIVERSITY
DEPARTMENT OF ELECTRICAL & ELECTRONICS ENGINEERING

ELE432

ADVANCED DIGITAL DESIGN

Homework Report - 2

25/04/2026

NAME AND SURNAME: RAHİME ESRA KARADENİZ

STUDENT ID: 21947325

1. Time Spent

I spent around 10 to 12 hours on this homework. Understanding the FSM state diagram took the most time. I had to go back to the lecture slides and the textbook several times to make sure I got the state transitions right. At first I made mistakes in the ALU decoder and the don't-care outputs. I spent a lot of time running the simulation, reading the error messages and figuring out what was wrong. The debugging process alone took several hours because I had to trace each failing test vector back to the exact signal that was wrong. I also had to understand how the testbench compares outputs so I could fix both the code and the test vectors correctly.

2. Test Results

When I first ran the simulation, I got 20 errors out of 40 tests, which is shown in the first screenshot below. All 20 errors were in the R-type instruction tests. The error messages showed that the expected value had an "X" in the ImmSrc position but my code was outputting "0". For example, the testbench expected "Xa2c" but my outputs were "0a2c". At first I thought the problem was in my controller code, so I changed the ImmSrc output for R-type from "xx" to "00" in the instrdec module. This made sense because the homework document says to set all don't-care outputs to 0. However, after making that change, I still got 20 errors. This time the testbench was expecting "X" but my code was outputting "0". The problem was that the test vectors themselves still had "XX" written in the ImmSrc column for all R-type instructions. The testbench reads these vectors and compares them bit by bit. When the vector says "X", the testbench treats it as a don't-care and skips that bit. But the homework said outputs should be 0, not X. So I needed to update both the code and the test vectors at the same time. I updated the test vectors by replacing all "XX" values in the ImmSrc column with "00" for R-type instructions. After making both changes together, the simulation passed all 40 tests with 0 errors. The reason the first run passed 40/40 was that the original test vectors used "X" as don't-care, which matched any output including "xx". Once I changed the test vectors to expect "00", my code also had to output "00" to match. This debugging process taught me that the test vectors and the design code must be consistent with each other.

```
VSIM9> run 2000ns
# Error on vector      0: inputs: op = 33 funct3 = 0 funct7b5 = 0; outputs = 0a2c (Xa2c expected)
# Error on vector      1: inputs: op = 33 funct3 = 0 funct7b5 = 0; outputs = 1420 (X420 expected)
# Error on vector      2: inputs: op = 33 funct3 = 0 funct7b5 = 0; outputs = 2020 (X020 expected)
# Error on vector      3: inputs: op = 33 funct3 = 0 funct7b5 = 0; outputs = 0022 (X022 expected)
# Error on vector      4: inputs: op = 33 funct3 = 0 funct7b5 = 1; outputs = 0a2c (Xa2c expected)
# Error on vector      5: inputs: op = 33 funct3 = 0 funct7b5 = 1; outputs = 1420 (X420 expected)
# Error on vector      6: inputs: op = 33 funct3 = 0 funct7b5 = 1; outputs = 2060 (X060 expected)
# Error on vector      7: inputs: op = 33 funct3 = 0 funct7b5 = 1; outputs = 0022 (X022 expected)
# Error on vector      8: inputs: op = 33 funct3 = 6 funct7b5 = 0; outputs = 0a2c (Xa2c expected)
# Error on vector      9: inputs: op = 33 funct3 = 6 funct7b5 = 0; outputs = 1420 (X420 expected)
# Error on vector     10: inputs: op = 33 funct3 = 6 funct7b5 = 0; outputs = 2010 (X010 expected)
# Error on vector     11: inputs: op = 33 funct3 = 6 funct7b5 = 0; outputs = 0022 (X022 expected)
# Error on vector     12: inputs: op = 33 funct3 = 7 funct7b5 = 0; outputs = 0a2c (Xa2c expected)
# Error on vector     13: inputs: op = 33 funct3 = 7 funct7b5 = 0; outputs = 1420 (X420 expected)
# Error on vector     14: inputs: op = 33 funct3 = 7 funct7b5 = 0; outputs = 2000 (X000 expected)
# Error on vector     15: inputs: op = 33 funct3 = 7 funct7b5 = 0; outputs = 0022 (X022 expected)
# Error on vector     16: inputs: op = 33 funct3 = 2 funct7b5 = 0; outputs = 0a2c (Xa2c expected)
# Error on vector     17: inputs: op = 33 funct3 = 2 funct7b5 = 0; outputs = 1420 (X420 expected)
# Error on vector     18: inputs: op = 33 funct3 = 2 funct7b5 = 0; outputs = 2070 (X070 expected)
# Error on vector     19: inputs: op = 33 funct3 = 2 funct7b5 = 0; outputs = 0022 (X022 expected)
# 40 tests completed with      20 errors
# ** Note: $stop      : controller_testbench.sv(81)
# Time: 415 ns Iteration: 1 Instance: /testbench
# Break in Module testbench at controller_testbench.sv line 81
VSIM10> quit -sim
```

The second screenshot right below shows the Questa transcript after running the simulation right after the debugging process.

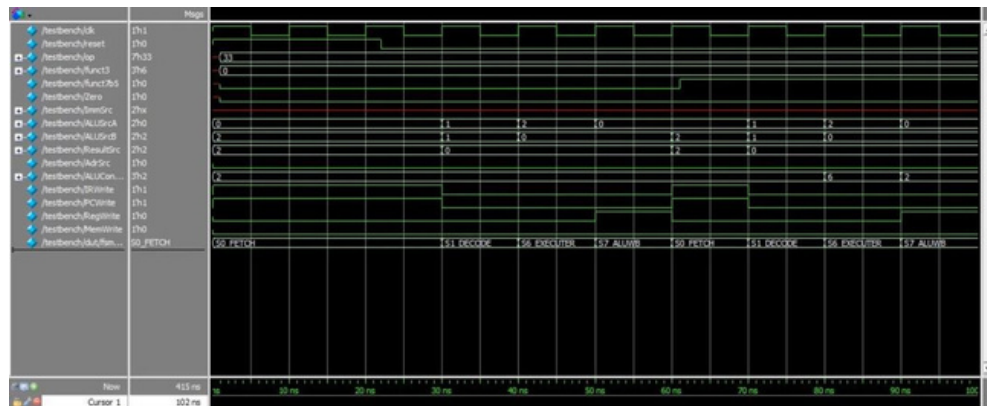
```
VSIM52> wave zoom range 130ns 210ns
# (130 ns) (210 ns)
VSIM53> wave zoom range 0ns 100ns
# (0 ns) (100 ns)
VSIM54> restart -f
# ** Note: (vsim-12125) Error and warning message counts have been reset to '0' because of 'restart'.
# ** Note: (vsim-9009) Loading existing optimized design _opt2
# ** Note: (vsim-12126) Error and warning message counts have been restored: Errors=0, Warnings=1.
# Loading work.testbench(fast)
# Loading work.controller(fast)
# Loading work.mainem(fast)
# Loading work.aludec(fast)
# Loading work.instrdec(fast)
VSIM55> run 2000ns
# 40 tests completed with      0 errors
# ** Note: $stop      : controller_testbench.sv(81)
# Time: 415 ns Iteration: 1 Instance: /testbench
# Break in Module testbench at controller_testbench.sv line 81
VSIM56>
```

Simulation Waveforms

The waveforms below show the controller outputs during simulation.
All signals are in hexadecimal.

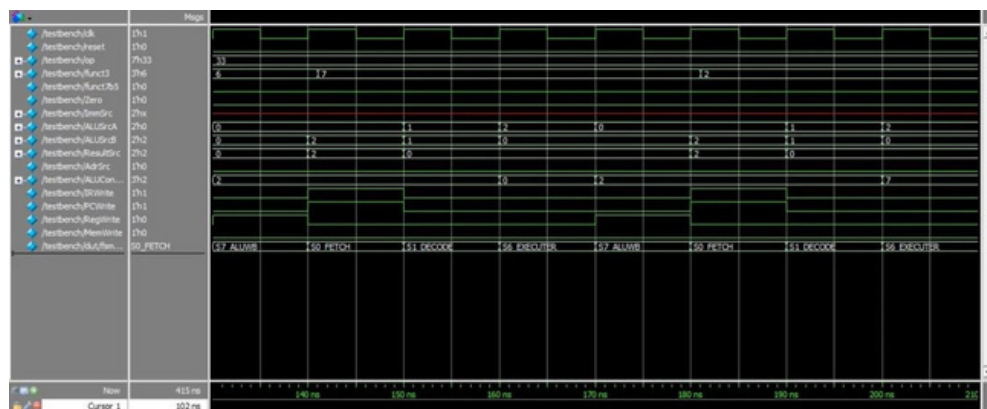
Waveform1 shows two full R-type instruction cycles (0 to 100 ns).

The state sequence is: S0 FETCH -> S1 DECODE -> S6 EXECUTER->S7 ALUWB -> S0 FETCH.
This is the correct 4-cycle sequence for R-type instructions like add, sub, or, and and slt.



Waveform2 shows later cycles (130 to 210 ns) with more R-type instructions.

The same S0 -> S1 -> S6 -> S7 pattern repeats. ALUControl changes based on funct3.
For example, ALUControl = 6 means subtract and ALUControl = 7 means SLT.



Key Signal Observations

Signal	What I observed
IRWrite	Goes high only during S0 FETCH as expected
PCWrite	Goes high during FETCH and JAL states
RegWrite	Goes high only during ALUWB and MemWB states
MemWrite	Goes high only during MemWrite state for sw
ALUSrcA	Changes between 0 1 and 2 depending on state
ALUSrcB	Changes between 0 1 and 2 depending on state
ALUControl	Changes based on instruction type and funct3
state (FSM)	Follows the correct sequence for each instruction type